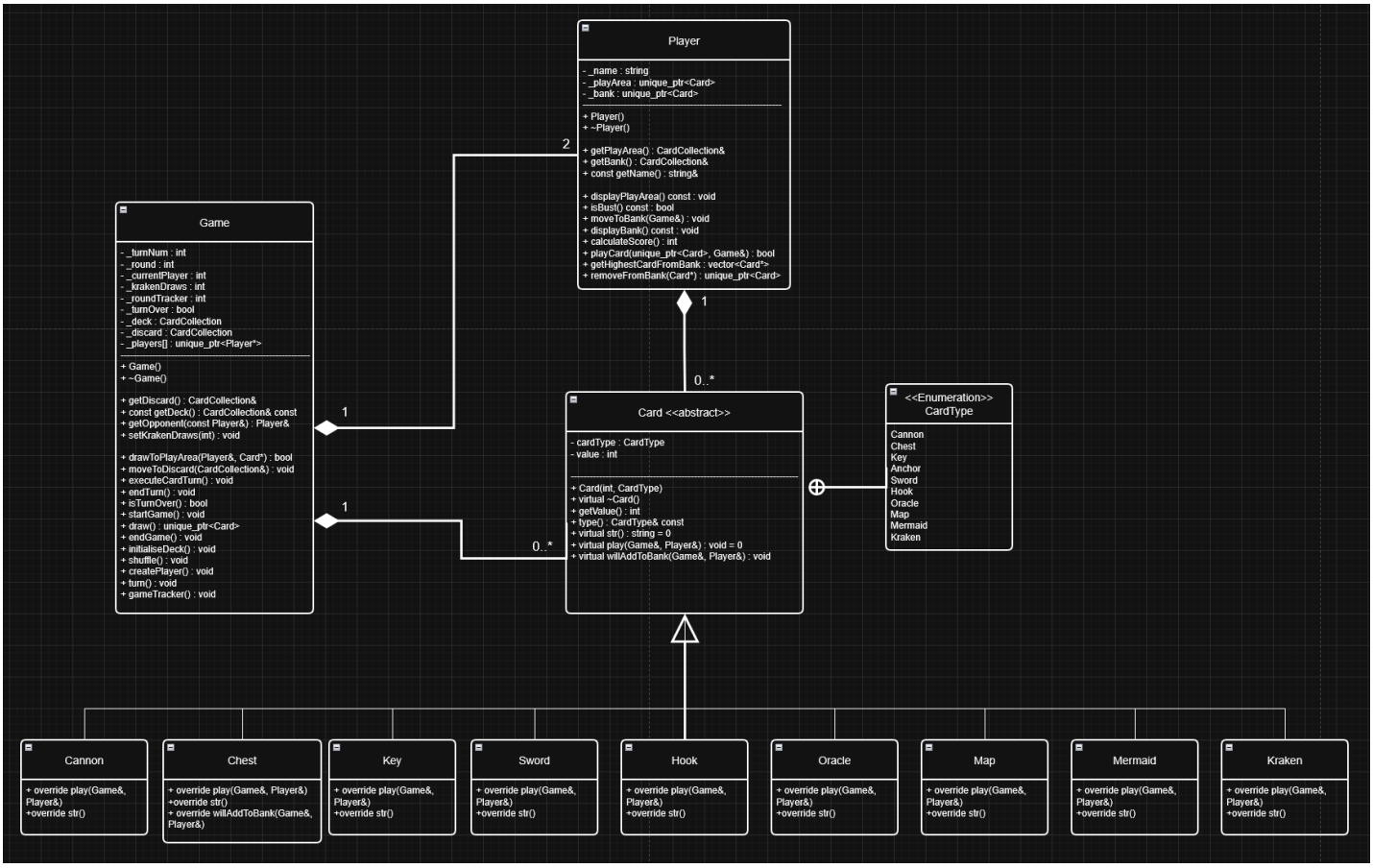
System Design Document

Class Diagram



How the Design Constraints Were Met:

DC1 - The required classes specified in the Specification must be implemented:

- A Game, Player, Card, and Card Abstract classes were implemented

DC2 - The Visual Studio 2022 solution you submit must compile and run:

- The game compiles, executes and runs as expected with no errors

DC3 - C++ standard library collections must be used for storing collections of objects:

- **std::map** was used to map cardtypes (suits) with their values and was used for grouping suits and displaying cards in the

bank, tracking the highest cards to calculate the score, and getting the top value cards from an opponent's bank
- **std::set** was implemented to check if the player is going to bust by first adding a card to the player's current play area and then creating a set and checking if any duplicates occur (since sets can't hold duplicates). This is more efficient (O(n)) than iterating through an array in $O(n^2)$ average time.
- **std::vector** was used to hold the collection of cards in the deck, discard, play area, and bank
- **Arrays** were used by fixing the number of player names to be randomly chosen from and to fix the number of players in the game.

DC4 - You must dynamically allocate your Card and Player objects:

- Unique pointers were used to reference each created card and player
- References to objects through functions were used throughout the entire project
- Utilised std::move and .get() functionality for unique pointers

DC5 - Dynamically allocated memory must be cleaned up correctly:

- Since unique pointers delete when they go out of scope, there wasn't any need for manual pointer deletion and thus there should not be any memory leaks

DC6 - Collections of pointers must be cleaned up correctly:

- See above

DC7 - Class member functions should be declared const unless they cannot be:

- Most getter functions were declared const, except for getDiscard(), getBank(), and getPlayArea(), since they were used and modified through card abilities like Map and Chest
- Any method that did not modify any data was declared const, such as displayBank/PlayArea(), and isBust().

DC8 - Initialiser lists must be used for constructors where initialisation is required:

- Initialiser lists were used in the Game and Card constructor to initialise all member variables

DC9 - Use code style guide for code formatting:

- All formatting rules were followed
- All header files implemented #ifndef, #define, and #endif
- All argument names are informative
- Naming conventions were followed
- Underscore was used for private member variables
- An enum class was used to define the card types
- The class methods have made use of private access sections so that the main file only needs a game object and the *startGame()* method. Methods that do not need to be public are private.

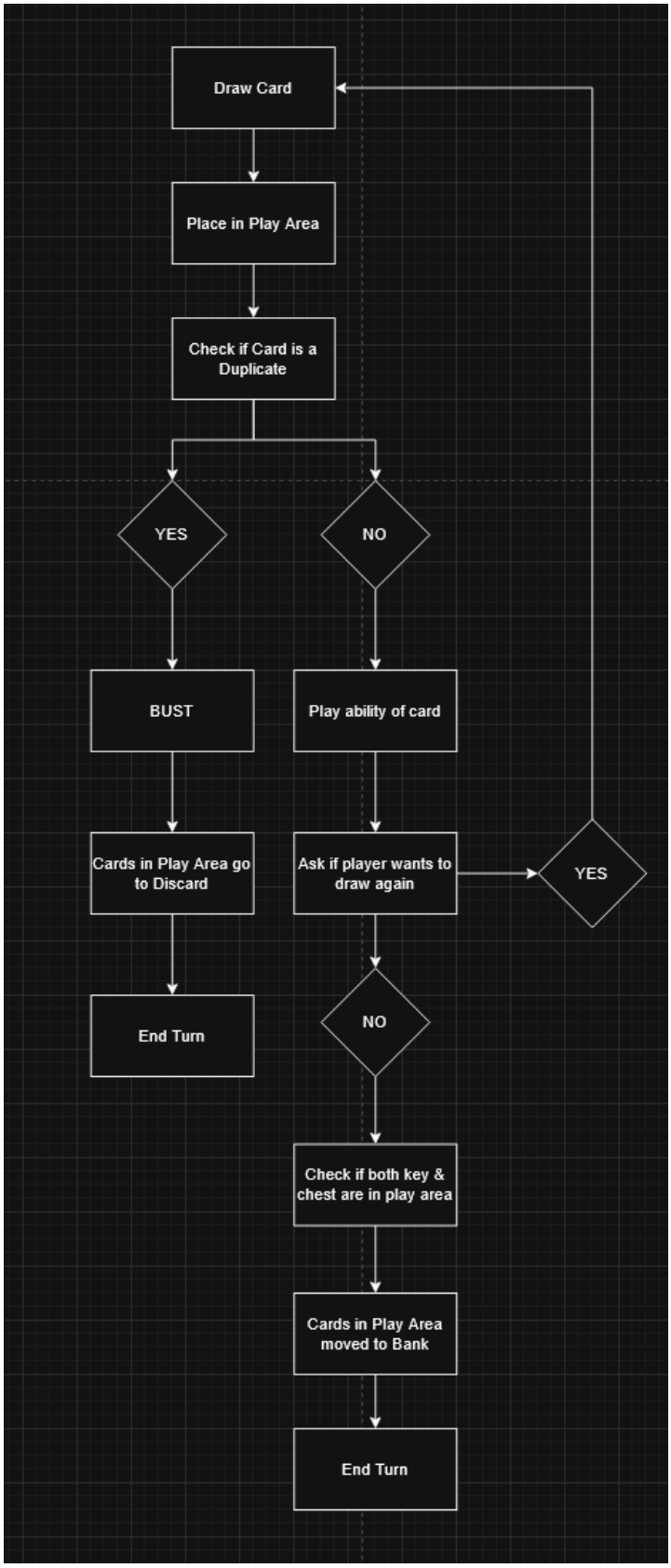
DC10 - Your system must not use external libraries:

- No external libraries were used

Other Design Implementation Choices

- The discard pile acts as a stack, so cards that are discarded will appear in the same order when an ability that sees the discard pile is used (Map for instance)
- For the Kraken ability, added a separate draw loop within the turn function in Game using an int krakenDraws variable to prevent duplicating the same code in the Kraken class
- The WillAddToBank function in Card is overridden in the Chest class with the Chest/Key ability functionality, so that ability is only run from one class to prevent duplication
- Though not specifically stated in the assignment spec, input handling is implemented so that no crashes/errors occur, and will continually ask the player to input again until it is correct
- Added an input at the very end of the game to prevent the game from quitting automatically, which prevents you from seeing the winner of the game

Flowchart of Gameplay Loop



Flow of the gameplay to provide visualisation to what is occurring, and shows the order of actions